



Projet SLMs

Explorer les **Small Language Models**
du concept à l'implémentation & au benchmark



Plan de la présentation

- 1 Contexte & Motivations
- 2 Architecture du Projet
- 3 Phase 1 — Ollama & les SLMs
- 4 Phase 2 — API FastAPI
- 5 Phase 3 — Benchmark des SLMs
- 6 Concepts Clés à Apprendre
- 7 Planning & Livrables
- 8 Conclusion

Pourquoi ce projet ?

💡 Constat de départ

Nous utilisons tous des **Large Language Models (LLMs)** au quotidien :

- **ChatGPT** (OpenAI)
- **Gemini** (Google)
- **Claude** (Anthropic)
- **Copilot** (Microsoft)

Mais ces modèles sont **hébergés dans le cloud** et nécessitent une **connexion** et des **coûts API**.

Explorer les **Small Language Models (SLMs)** qui peuvent tourner **localement** sur votre machine !

- ✓ Pas de frais API
- ✓ Données privées
- ✓ Faible latence
- ✓ Personnalisable

LLMs vs SLMs — Comprendre la différence



LLMs (Large Language Models)

- Des **milliards** de paramètres (70B+)
- Nécessitent des **serveurs puissants** (GPU A100, H100)
- Accès via **API payante**
- Capacités très larges et générales
- Ex : GPT-4, Claude 3, Gemini Ultra



SLMs (Small Language Models)

- **Quelques milliards** de paramètres (1B–13B)
- Tournent sur **votre laptop** (RAM 8–16 GB)
- **Gratuits** et open source
- Spécialisables sur des tâches précises
- Ex : **Mistral 7B, LLaMA 3, Phi-3, Gemma**

Pour ce projet : vous allez faire tourner des SLMs **en local** grâce à **Ollama**, les exposer via **FastAPI** et les **comparer objectivement**.

Les 2 grandes parties du projet

1 — Backend IA avec FastAPI

- Installer **Ollama** & télécharger des SLMs
- Créer une **API REST** avec **FastAPI**
- Implémenter les tâches NLP (extraction, résumé, Q&A...)
- Gérer le **streaming** des réponses
- Tester avec **Swagger UI** intégré

2 — Benchmark & Évaluation des SLMs

- Définir des **métriques** d'évaluation
- Comparer **vitesse**, **qualité** et **consommation mémoire**
- Tester plusieurs modèles sur les **mêmes tâches**
- Produire des **rapports visuels** (graphiques, tableaux)
- Identifier le meilleur modèle **pour chaque usage**

Qu'est-ce qu'Ollama ?

Ollama en bref

Ollama est un outil open source qui permet de faire tourner des SLMs **localement** en quelques commandes.

- Installation simple sur **Windows / Mac / Linux**
- Gère le téléchargement des modèles
- Expose une **API REST locale** (port 11434)
- Compatible avec des dizaines de modèles

```
# Installer Ollama
curl -fsSL
https://ollama.com/install.sh |
sh

# Télécharger un modèle
ollama pull mistral

# Lancer une conversation
ollama run mistral

# Lister les modèles
ollama list
```

SLMs disponibles — Panorama complet

★ Modèles recommandés

Modèle	Params	RAM	Point fort
mistral	7B	8 GB	Généraliste
llama3.2	3B	4 GB	Rapide
phi3	3.8B	4 GB	Raisonnement
gemma2	2B	3 GB	Léger
qwen2.5	7B	8 GB	Multilingue
deepseek-r1	7B	8 GB	CoT avancé

💡 Stratégie de test

Choisissez **3 modèles** de tailles différentes et testez-les sur les **mêmes prompts** :

- Un modèle **petit** (2B–3B) pour la vitesse
- Un modèle **moyen** (7B) pour la qualité
- Un modèle **spécialisé** (code, multilang...)

Le but : trouver le **meilleur compromis** pour chaque tâche.

Anatomie d'un appel Ollama

🔗 Appel direct à l'API Ollama (Python)

```
1 import requests, json, time
2
3 def call_ollama(model: str, prompt: str) -> dict:
4     start = time.perf_counter()
5
6     response = requests.post(
7         "http://localhost:11434/api/generate",
8         json={
9             "model": model,
10            "prompt": prompt,
11            "stream": False,
12            "options": {"temperature": 0.2, "num_predict": 512}
13        }
14    )
15    data = response.json()
16    elapsed = time.perf_counter() - start
17
18    return {
19        "model": model,
```

Pourquoi FastAPI ?

⚡ Avantages de FastAPI

- **Swagger UI** généré automatiquement — testez vos routes sans Postman
- **Validation** automatique des données avec **Pydantic**
- Support natif du **streaming** (Server-Sent Events)
- **Async/await** pour gérer plusieurs requêtes en parallèle
- **Typage** Python strict — moins de bugs
- Performances proches de **Node.js** / **Go**

```
# Installer les dépendances
pip install fastapi uvicorn
httpx pydantic

# Lancer le serveur
uvicorn main:app --reload

# Swagger UI disponible sur
http://localhost:8000/docs

# Redoc disponible sur
http://localhost:8000/redoc
```

Structure de l'API FastAPI

main.py — Squelette de l'API

```
1 from fastapi import FastAPI
2 from pydantic import BaseModel
3 import httpx, json
4
5 app = FastAPI(title="SLM_API", version="1.0")
6
7 OLLAMA_URL = "http://localhost:11434/api/generate"
8
9 class NLPRequest(BaseModel):
10     text: str
11     model: str = "mistral"
12     task: str = "summarize" # summarize | extract | classify | qa
13
14 class NLPResponse(BaseModel):
15     result: str | dict
16     model: str
17     tokens_per_sec: float
18     latency_s: float
19
```

Tâches NLP à implémenter

Tâches de base

- **Résumé** d'un article ou document
- **Extraction d'entités** (personnes, entreprises, chiffres)
- **Classification** de documents par thème
- **Mots-clés** thématiques en JSON

- **Q&A** sur un document (RAG basique)
- **Sentiment analysis** (positif/négatif/neutre)
- **Traduction** ou reformulation
- **Génération** de résumé exécutif structuré
- **Détection de langue**

⚠ Important : Chaque tâche doit être un endpoint **séparé** avec sa propre logique de prompt. Commencez par le résumé, puis itérez.

Streaming avec FastAPI

☰ Réponse en streaming (Server-Sent Events)

```
1 from fastapi.responses import StreamingResponse
2 import httpx, json
3
4 @app.post("/nlp/stream")
5 async def stream_response(req: NLPRequest):
6     async def token_generator():
7         async with httpx.AsyncClient() as client:
8             async with client.stream("POST", OLLAMA_URL,
9                                     json={"model": req.model, "prompt": req.text,
10                                           "stream": True}) as r:
11                 async for line in r.aiter_lines():
12                     if line:
13                         chunk = json.loads(line)
14                         yield chunk.get("response", "")
15                         if chunk.get("done"):
16                             break
17
18     return StreamingResponse(token_generator(),
19                             media_type="text/event-stream")
```

Pourquoi benchmarker les SLMs ?

L'enjeu du benchmark

Tous les SLMs ne se valent pas. Un benchmark rigoureux permet de :

- Choisir le modèle **optimal** pour chaque tâche
- Quantifier le **trade-off** vitesse vs qualité
- Identifier les **limites** de chaque modèle
- Justifier un choix technique par des **données objectives**
- Simuler des conditions **réelles de production**

Performance :

- Tokens / seconde
- Latence (Time to First Token)
- RAM utilisée (Mo)

Qualité :

- Score humain (1–5)
- Cohérence JSON (parse OK ?)
- BLEU / ROUGE (si référence)
- Taux d'hallucination

Corpus de test — Comment le construire ?

Structure d'un corpus de test

Chaque échantillon doit contenir :

- Le **texte d'entrée** (article, rapport, question)
- La **tâche** à effectuer
- Une **référence** (réponse attendue, si possible)
- Des **métadonnées** (langue, domaine, longueur)

Visez **10–20 échantillons** par tâche pour des résultats significatifs.

- **Articles courts** (< 200 mots) vs **longs** (> 1000 mots)
- Textes en **français, anglais, arabe**
- Domaines variés : finance, médical, actualité, technique
- Cas **limites** : texte vide, très court, ambigu
- Questions **factuelles** vs **inférentielles**

Évaluation qualitative — Grille de notation



Grille d'évaluation humaine

Pour chaque réponse générée, notez de **1 à 5** selon ces critères :

Pertinence (1–5)

La réponse répond-elle bien à la tâche ?

Précision (1–5)

Les informations sont-elles exactes ?

Format (1–5)

Le format demandé (JSON, liste. . .) est-il respecté ?

Concision (1–5)

La réponse est-elle trop longue / trop courte ?

Hallucination (oui/non)

Le modèle a-t-il inventé des faits ?

Score global = moyenne des 4 critères

→ Exportez en CSV pour comparer

Exemple de résultats attendus

🗪 Tableau comparatif typique

Modèle	Tok/s	Latence	RAM	Qualité	JSON OK	Score
Gemma 2B	45	1.2s	3 GB	3.1	80%	Rapide
Phi-3 3.8B	28	1.8s	4 GB	3.8	90%	Équilibré
Mistral 7B	14	3.5s	8 GB	4.5	95%	Qualité
Qwen2.5 7B	12	4.0s	8 GB	4.3	92%	Multilingue

❗ Les chiffres ci-dessus sont **indicatifs**. Vos résultats dépendront de votre machine et de vos prompts. C'est l'intérêt du benchmark !

Bonnes pratiques — Prompt Engineering

❌ Mauvais prompt

"Extrais les infos de ce texte : [texte]"



"Tu es un expert en analyse financière.
Extrait du texte suivant UNIQUEMENT :
- Les entités nommées (entreprises, personnes)
- Les chiffres financiers avec leurs unités
- La date de publication
Réponds en JSON valide avec les clés :
'entities', 'figures', 'date'.
Texte : [texte]"



Règles d'or du prompting

- 1 Donnez un **rôle** au modèle
- 2 Soyez **très précis** sur le format de sortie
- 3 Utilisez des **exemples** (few-shot)
- 4 Décomposez les tâches complexes (**chain of thought**)
- 5 Testez et **itérez** sur vos prompts
- 6 Spécifiez la **langue** de réponse

i Le même prompt peut donner des résultats très **différents** selon le modèle — c'est un axe de benchmark en soi !

Ressources & Concepts à maîtriser



Concepts fondamentaux

- **LLM / SLM** — architectures Transformer
- **Quantization** — INT4 / INT8 pour réduire la RAM
- **Prompt Engineering** — few-shot, CoT, role prompting
- **RAG** — enrichir le modèle avec vos données
- **Embeddings** — représentation vectorielle du texte
- **GGUF** — format de modèle optimisé pour Ollama
- **Temperature / Top-P** — contrôler la créativité



Ressources recommandées

- **Andrej Karpathy** — « Let's build GPT » (YouTube)
- **3Blue1Brown** — Neural Networks series
- **Hugging Face** — cours NLP gratuit (huggingface.co/learn)
- **FastAPI docs** — fastapi.tiangolo.com
- **Ollama docs** — ollama.com/docs
- **LangChain docs** — orchestration LLM avancée
- **Papers With Code** — derniers benchmarks

Planning suggéré du projet

Semaines 1–2 : Découverte

-  Vidéos sur LLMs, SLMs, Transformers
-  Installer Ollama, tester 3 modèles en CLI
-  Premiers appels à l'API REST Ollama
Rédiger vos premières observations

-  Construire le corpus de test
Mesurer vitesse, RAM, qualité
-  Évaluation humaine des réponses
-  Générer graphiques et tableaux comparatifs

Semaines 3–4 : API FastAPI

-  Créer le projet FastAPI
Implémenter 3+ tâches NLP
-  Ajouter le streaming

-  Démo complète de l'API
Rapport benchmark final
-  Expérimentation RAG (bonus)

Code à produire

1 API FastAPI

- Endpoints documentés (Swagger auto)
- Au moins 3 tâches NLP
- Support du streaming
- Tests unitaires (pytest)

2 Module de Benchmark

- Script automatisé multi-modèles
- Export CSV + graphiques PNG
- Corpus de test versionné

Documents à produire

1 README (installation, usage, exemples)

2 Rapport de benchmark incluant :

- Tableau comparatif des SLMs
- Graphiques vitesse / qualité / RAM
- Analyse par tâche NLP
- Recommandations finales
- Pistes : RAG, fine-tuning, quantization

3 Présentation démo finale

Ce que vous allez acquérir



Compétences techniques

- Maîtrise des **SLMs en local** (Ollama)
- Développement d'API avec **FastAPI** & Python async
- **Prompt Engineering** avancé
- Méthodologie de **benchmark** rigoureuse
- Bases du **RAG** et des **embeddings**
- Analyse de données avec **Pandas** & **Matplotlib**



Compétences transversales

- **Veille technologique** active (AI & ML)
- Capacité à **évaluer objectivement** des modèles IA
- **Documentation** technique de qualité
- Mindset **itératif** et expérimental
- **Résolution de problèmes** complexes
- Comprendre les **limites réelles** des SLMs

Ce projet vous donnera une base solide pour comprendre, déployer et évaluer l'IA générative dans un contexte professionnel réel.



C'est à vous de jouer !

Bonne chance à toute l'équipe stagiaires



Département IA 15 juin 2026